<table>
<tr><td colspan="2">Equipo 7:

Luis Enrique Almaguer Castillo - 2143523

Héctor Alexander Tovar González - 2144664

Lizzie Anette Mireles Tejada - 2144591</td></tr>
<tr><td>Nombre del curso: Componentes de lógica de negocio</td><td>Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td>Módulo Netacad: U-5</td><td>Actividad: ACT05-Laboratorio guiado.</td></tr>
<tr><td colspan="2">Fecha: 05/11/2026</td></tr>
<tr><td colspan="2">Repositorio de GitHub: https://github.com/Lamaguer/NovaRent-U5</td></tr>
</table>

Objetivo: Seguir los pasos del laboratorio para realizar una cobertura mayor al 80% y automatizar las pruebas con GitHUb actions, de esta manera poder expresar estos conocimientos dentro del Producto Integrador de Aprendizaje.

Evidencia 1

```
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio> cd Proyecto_Python_Base_U5
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5> python -m pip insta
-r requirements.txt
Collecting pytest<10,>=8 (from -r requirements.txt (line 1))
  Downloading pytest-9.2.0-py3-none-any.whl.metadata (7.9 kB)
Collecting pytest-cov<8,>=5 (from -r requirements.txt (line 2))
  Downloading pytest_cov-7.1.0-py3-none-any.whl.metadata (32 kB)
Collecting SQLAlchemy<3,>=2 (from -r requirements.txt (line 3))
  Downloading sqlalchemy-2.1.6-cp314-cp314-win_amd64.whl.metadata (9.9 kB)
Collecting ruff>=0.6 (from -r requirements.txt (line 4))
  Downloading ruff-0.17.3-py3-none-win_amd64.whl.metadata (21 kB)
Collecting colorama>=0.4 (from pytest<10,>=8->-r requirements.txt (line 1))
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting iniconfig>=2 (from pytest<10,>=8->-r requirements.txt (line 1))
  Using cached iniconfig-2.3.1-py3-none-any.whl.metadata (2.8 kB)
Collecting packaging>=24 (from pytest<10,>=8->-r requirements.txt (line 1))
  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.6 (from pytest<10,>=8->-r requirements.txt (line 1))
  Using cached pluggy-1.7.0-py3-none-any.whl.metadata (5.0 kB)
Collecting pygments>=2.15 (from pytest<10,>=8->-r requirements.txt (line 1))
  Using cached pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)
Collecting coverage>=7.10.6 (from coverage[toml]>=7.10.6->pytest-cov<8,>=5->-r requirements.txt (line 2))
  Downloading coverage-7.17.0-cp314-cp314-win_amd64.whl.metadata (8.3 kB)
Collecting typing-extensions>=4.6.0 (from SQLAlchemy<3,>=2->-r requirements.txt (line 3))
  Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Downloading pytest-9.2.0-py3-none-any.whl (411 kB)
Downloading pytest_cov-7.1.0-py3-none-any.whl (22 kB)
Downloading sqlalchemy-2.1.6-cp314-cp314-win_amd64.whl (2.4 MB)
   ---------------------------------------- 2.4/2.4 MB 4.6 MB/s  0:00:00
Using cached pluggy-1.7.0-py3-none-any.whl (27 kB)
Downloading ruff-0.17.3-py3-none-win_amd64.whl (10.3 MB)
   ---------------------------------------- 10.3/10.3 MB 2.9 MB/s  0:00:03
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading coverage-7.17.0-cp314-cp314-win_amd64.whl (228 kB)
Using cached iniconfig-2.3.1-py3-none-any.whl (7.6 kB)
Using cached packaging-26.3-py3-none-any.whl (129 kB)
Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)
```

```
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading coverage-7.17.0-cp314-cp314-win_amd64.whl (228 kB)
Using cached iniconfig-2.3.1-py3-none-any.whl (7.6 kB)
Using cached packaging-26.3-py3-none-any.whl (129 kB)
Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)
Using cached typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Installing collected packages: typing-extensions, ruff, pygments, pluggy, packaging, iniconfig, coverage, colorama, SQLA
lchemy, pytest, pytest-cov
Successfully installed SQLAlchemy-2.1.6 colorama-0.4.6 coverage-7.17.0 iniconfig-2.3.1 packaging-26.3 pluggy-1.7.0 pygme
nts-2.21.0 pytest-9.2.0 pytest-cov-7.1.0 ruff-0.17.3 typing-extensions-4.16.0

[notice] A new release of pip is available: 26.2.1 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5> python -m pytest -v
============================================= test session starts =============================================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laborat
orio\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 13 items

tests/test_order.py::test_vip_discount PASSED                                                        [  7%]
tests/test_order.py::test_regular_no_discount PASSED                                                 [ 15%]
tests/test_order.py::test_customer_debt PASSED                                                       [ 23%]
tests/test_order.py::test_invalid_total[0] PASSED                                                    [ 30%]
tests/test_order.py::test_invalid_total[-1] PASSED                                                   [ 38%]
tests/test_repository.py::test_repository_commit PASSED                                              [ 46%]
tests/test_repository.py::test_repository_rollback PASSED                                            [ 53%]
tests/test_security.py::test_owner_within_24h PASSED                                                 [ 61%]
tests/test_security.py::test_stranger_denied PASSED                                                  [ 69%]
tests/test_security.py::test_shipped_denied_even_admin PASSED                                        [ 76%]
tests/test_security.py::test_expired_denied PASSED                                                   [ 84%]
tests/test_service.py::test_gateway_approved PASSED                                                  [ 92%]
tests/test_service.py::test_gateway_rejected PASSED                                                  [100%]

============================================== 13 passed in 1.92s ==============================================
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5>
```

¿Para qué sirve Pytest? - Pytest sirve para comprobar que el código de Python que has hecho hasta el momento funciona de verdad con pruebas creadas por el programador para simular un funcionamiento real de lo que se supone que tiene que hacer el código.

¿Para qué sirve el Entorno Virtual? - El entorno visual se podría decir que es un pequeño espacio en la computadora donde se instalan todas las librerías necesarias para un proyecto, pero solo se instalan ahí para evitar interferencias con algún otro proyecto en la computadora.

Evidencia 2

```python
#Prueba con total de 200 buckaroos y Cliente VIP, resultado esperado 170.
def test_vip_discount2():
    o = Order(200, vip=True)
    assert o.confirm() == 170
    assert o.status == "CONFIRMED"

def test_regular_no_discount():
    assert Order(100).confirm() == 100

#Prueba con total de 200 buckaroos y Cliente regular, Resultado esperado 200.
def test_regular_no_discount2():
    assert Order(200).confirm() == 200
```

```
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5> python -m pytest tests/test_order.py -v
============================================= test session starts =============================================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5
configfile: pyproject.toml
plugins: cov-7.1.0
collected 7 items

tests/test_order.py::test_vip_discount PASSED                                                        [ 14%]
tests/test_order.py::test_vip_discount2 PASSED                                                       [ 28%]
tests/test_order.py::test_regular_no_discount PASSED                                                 [ 42%]
tests/test_order.py::test_regular_no_discount2 PASSED                                                [ 57%]
tests/test_order.py::test_customer_debt PASSED                                                       [ 71%]
tests/test_order.py::test_invalid_total[0] PASSED                                                    [ 85%]
tests/test_order.py::test_invalid_total[-1] PASSED                                                   [100%]

============================================== 7 passed in 0.20s ==============================================
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5>
```

Nombre Prueba	Entrada	Regla	Resultado Esperado	Resultado Real
test_regular_no_discount 2()	total = 200 VIP = False	Cliente VIP obtiene un 15% de descuento.	Que el total siga siendo 200 por no tener VIP.	Prueba pasada, el resultado fue 200.
test_vip_discount2()	total = 200 VIP = True	Cliente Regular no obtiene ningún descuento.	Que el total sea 170 por tener VIP.	Prueba pasada, el resultado fue 170.

Evidencia 3

RED

```
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5> python -m pytest tests/test_order.py -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5
configfile: pyproject.toml
plugins: cov-7.1.0
collected 8 items

tests/test_order.py::test_vip_discount PASSED                            [ 12%]
tests/test_order.py::test_vip_discount2 PASSED                           [ 25%]
tests/test_order.py::test_regular_no_discount PASSED                     [ 37%]
tests/test_order.py::test_regular_no_discount2 PASSED                    [ 50%]
tests/test_order.py::test_customer_debt PASSED                           [ 62%]
tests/test_order.py::test_invalid_total[0] PASSED                        [ 75%]
tests/test_order.py::test_invalid_total[-1] PASSED                       [ 87%]
tests/test_order.py::test_cannot_confirm_twice FAILED                    [100%]

================================== FAILURES ===================================
_____________________________ test_cannot_confirm_twice _____________________________

    def test_cannot_confirm_twice():
        order = Order(100)
        order.confirm()
>       with pytest.raises(ValueError, match="already confirmed"):
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       Failed: DID NOT RAISE ValueError

tests\test_order.py:34: Failed
```

GREEN

```
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5> python -m pytest tests/test_order.py::test_cannot_confirm_twice -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

tests/test_order.py::test_cannot_confirm_twice PASSED                    [100%]

============================== 1 passed in 0.16s ==============================
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5> python -m pytest tests/test_order.py -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5
configfile: pyproject.toml
plugins: cov-7.1.0
collected 8 items

tests/test_order.py::test_vip_discount PASSED                            [ 12%]
tests/test_order.py::test_vip_discount2 PASSED                           [ 25%]
tests/test_order.py::test_regular_no_discount PASSED                     [ 37%]
tests/test_order.py::test_regular_no_discount2 PASSED                    [ 50%]
tests/test_order.py::test_customer_debt PASSED                           [ 62%]
tests/test_order.py::test_invalid_total[0] PASSED                        [ 75%]
tests/test_order.py::test_invalid_total[-1] PASSED                       [ 87%]
tests/test_order.py::test_cannot_confirm_twice PASSED                    [100%]

============================== 8 passed in 0.10s ==============================
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5>
```

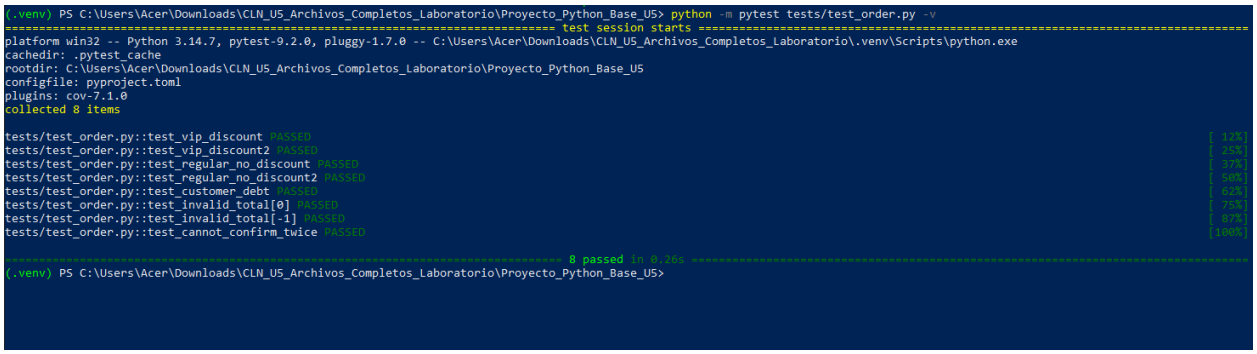
Fragmento de implementación:

```python
def _validate_confirmation(self, customer_debt=0):
    #Validacion se hace un metodo externo por REFRACTOR
    if self.status == "CONFIRMED":
        raise ValueError("Already confirmed.")
    if self.total <= 0:
        raise ValueError("total must be positive")
    if customer_debt > 0:
        raise CustomerHasDebt("outstanding debt")

def confirm(self, customer_debt=0):
    self._validate_confirmation(customer_debt)
    self.discount = round(self.total * 0.15, 2) if self.vip else 0.0
    self.status = "CONFIRMED"
    return self.total - self.discount
```

```python
def test_cannot_confirm_twice():
    order = Order(100)
    order.confirm()
    with pytest.raises(ValueError, match="Already confirmed."):
        order.confirm()
```

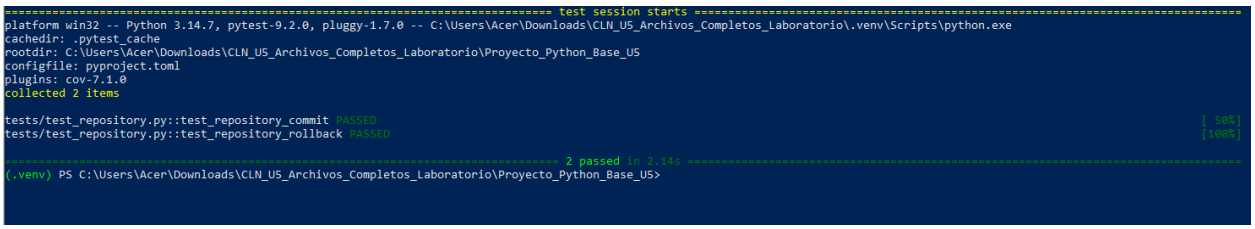
Pruebas después de Refractor



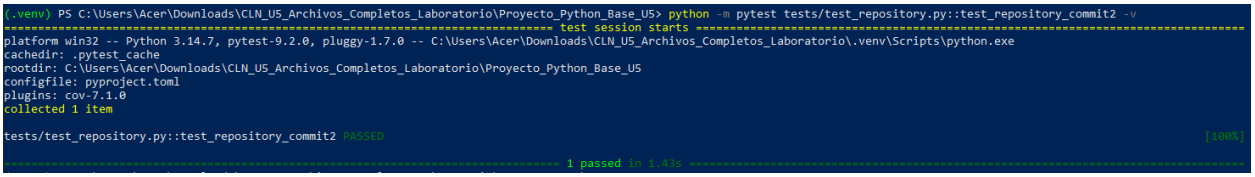
TDD no es solo ejecutar reglas después de programar, es crear las reglas antes de programar de esta manera se puede tener una idea esencial de que se tiene que hacer antes de que se haga, es una guía para el mismo programador facilitando saber que tiene que entrar y salir y de qué manera específicamente, TDD pide que laregla falle inicialmente, que eso es el concepto de RED para de esa forma confirmar que la nueva regla falta y después de esta falla hacer el código mínimo para que esta pueda pasar (green).

Evidencia 4

Respeta commits y rollbacks:



Segundo commit:



Codigo de prueba:

```python
#Prueba 2 total 250 con id==2
def test_repository_commit2():
    engine = make_engine()
    with Session(engine) as session:
        SqlOrderRepository(session).save(2, Order(250))
        session.commit()
        assert session.scalar(select(OrderRow).where(OrderRow.id == 2)).total == 250
# python -m pytest tests/test_repository.py::test_repository_commit2 -v
```

Diferencia entre commit y rollback - La diferencia entre estos dos es más clara que el agua el commit y el rollback hacen completamente lo contrario, el commit hace persistente un cambio temporal dentro de algo como sería la base de datos, añade una fila mientras el rollback deshace esos cambios pendientes, es un control z de las modificaciones temporales.

Evidencia 5

```
============================= test session starts ==============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5
configfile: pyproject.toml
plugins: cov-7.1.0
collected 2 items

tests/test_service.py::test_gateway_approved PASSED                      [ 50%]
tests/test_service.py::test_gateway_rejected PASSED                      [100%]

============================== 2 passed in 0.22s ==============================
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5> python -m pytest tests/test_service.py::test_gateway_rejected2 -v
============================= test session starts ==============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

tests/test_service.py::test_gateway_rejected2 PASSED                     [100%]

============================== 1 passed in 0.18s ==============================
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5> python -m pytest tests/test_service.py::test_gateway_rejected2 -v
============================= test session starts ==============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

tests/test_service.py::test_gateway_rejected2 PASSED                     [100%]

============================== 1 passed in 0.53s ==============================
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5>
```

Propuesta para evitar el CONFIRM de pagos no exitosos.

```python
class OrderService:
    def __init__(self, gateway):
        self.gateway = gateway

    def checkout(self, order):
        amount = order.total - round(order.total * 0.15, 2) if order.vip else order.total
        if not self.gateway.charge(amount):
            order.status = "REJECTED"
            raise RuntimeError("payment rejected")
        order.confirm()
        return amount
```

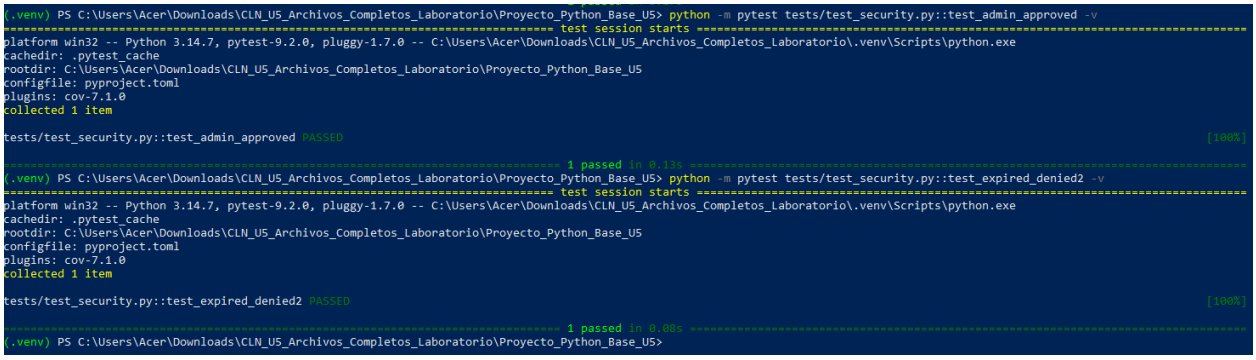
La orden se cancela si esta no tiene algún cargo (Si falla) y queda en rejected antes de hacer el raise, de otro modo se confirma. Entonces gracias a que existe el estatus REJECTED, ponemos hacer un assert de REJECTED con nuestra prueba de 200 y VIP.

```python
def test_gateway_rejected2():
    gateway = Mock()
    gateway.charge.return_value = False
    order = Order(200, vip=True)

    with pytest.raises(RuntimeError):
        OrderService(gateway).checkout(order)

    gateway.charge.assert_called_once_with(170)
    assert order.status == "REJECTED"
```

Evidencia 6



Test	Rol	¿Propietario?	Estado	Antiguedad	Resultado
test_owner_within_24h()	Usuario / Regular	Sí	NEW	Menos de 24 Horas	Permitido
test_stranger_denied()	Usuario / Regular	No	NEW	Menos de 24 Horas	Denegado
test_admin_approved()	Administrador	No	NEW	Cualquier antigüedad	Permitido
test_shipped_denied_even_admin()	Administrador	Sí o no	SHIPPED	Cuaquier antiguedad	Denegado
test_expired_denied2()	Usuario / Regular	Sí	NEW	Exactamente 24 Horas	Denegado
test_expired_denied()	Usuario / Regular	Sí	NEW	Mas de 24 Horas	Denegado

Evidencia 7

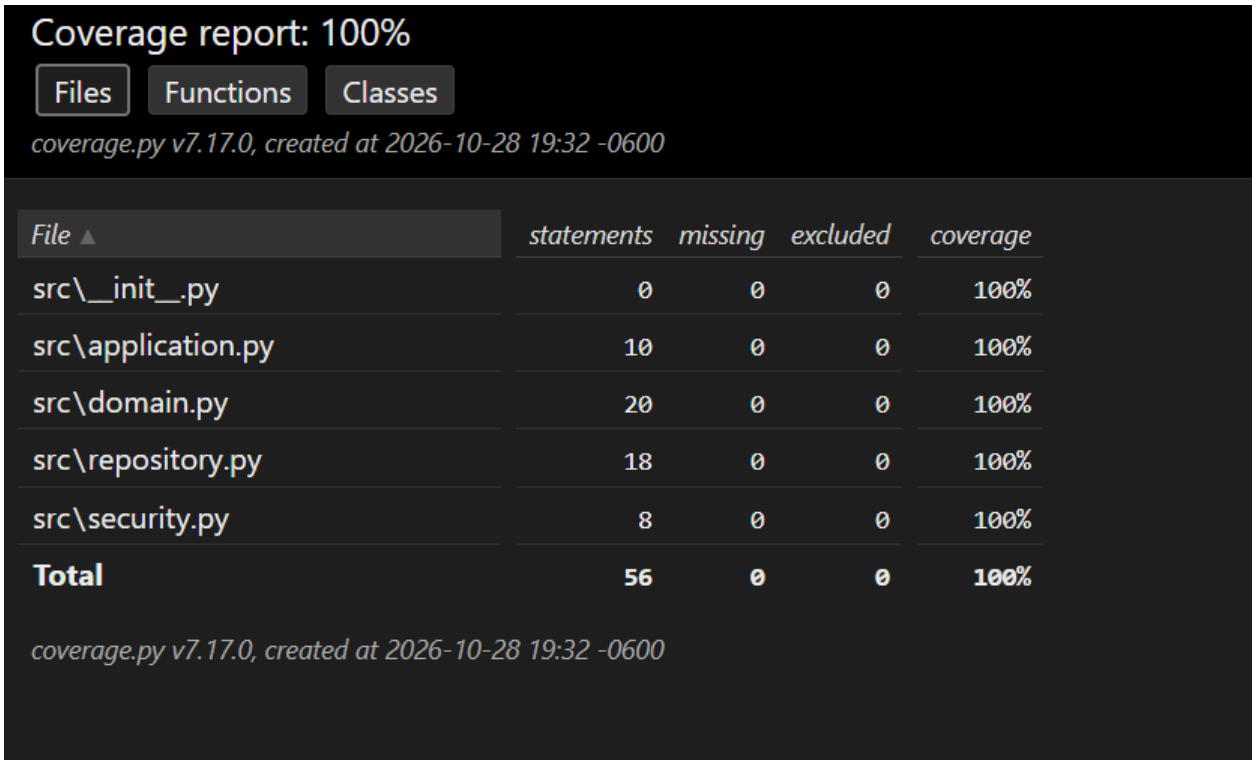
```
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5> python -m ruff check src tests
All checks passed!
```

```
(.venv) PS C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5> python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-u
nder=80
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\Proyecto_Python_Base_U5
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 20 items

tests/test_order.py::test_vip_discount PASSED                            [  5%]
tests/test_order.py::test_vip_discount2 PASSED                           [ 10%]
tests/test_order.py::test_regular_no_discount PASSED                     [ 15%]
tests/test_order.py::test_regular_no_discount2 PASSED                    [ 20%]
tests/test_order.py::test_customer_debt PASSED                           [ 25%]
tests/test_order.py::test_invalid_total[0] PASSED                        [ 30%]
tests/test_order.py::test_invalid_total[-1] PASSED                       [ 35%]
tests/test_order.py::test_cannot_confirm_twice PASSED                    [ 40%]
tests/test_repository.py::test_repository_commit PASSED                  [ 45%]
tests/test_repository.py::test_repository_commit2 PASSED                 [ 50%]
tests/test_repository.py::test_repository_rollback PASSED                [ 55%]
tests/test_security.py::test_owner_within_24h PASSED                     [ 60%]
tests/test_security.py::test_stranger_denied PASSED                      [ 65%]
tests/test_security.py::test_admin_approved PASSED                       [ 70%]
tests/test_security.py::test_shipped_denied_even_admin PASSED            [ 75%]
tests/test_security.py::test_expired_denied2 PASSED                      [ 80%]
tests/test_security.py::test_expired_denied PASSED                       [ 85%]
tests/test_service.py::test_gateway_approved PASSED                      [ 90%]
tests/test_service.py::test_gateway_rejected PASSED                      [ 95%]
tests/test_service.py::test_gateway_rejected2 PASSED                     [100%]C
:\Users\Acer\AppData\Local\Programs\Python\Python314\Lib\ast.py:46: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x0000028CFA7D9B70>
  return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Acer\AppData\Local\Programs\Python\Python314\Lib\ast.py:46: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x0000028CFA7DA020>
  return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback


---------- tests coverage ----------
_____ coverage: platform win32, python 3.14.7-final-0 _____

Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
src\__init__.py         0      0   100%
src\application.py      10      0   100%
src\domain.py           20      0   100%
src\repository.py       18      0   100%
src\security.py          8      0   100%
-------------------------------------------------
TOTAL                   56      0   100%
Coverage HTML written to dir htmlcov
Required test coverage of 80% reached. Total coverage: 100.00%
============================= 20 passed in 3.02s ==============================
C:\Users\Acer\Downloads\CLN_U5_Archivos_Completos_Laboratorio\.venv\Lib\site-packages\_pytest\unraisableexception.py:29: ResourceWarning: unclosed database in <sqlite3.Connection objec
t at 0x0000028CFA7ACF40>
  gc.collect()
```



Coverage report: 100%

Files | Functions | Classes

coverage.py v7.17.0, created at 2026-10-28 19:32 -0600

File ▲	function	statements	missing	excluded	coverage
src__init__.py	*(no function)*	0	0	0	100%
src\application.py	OrderService.__init__	1	0	0	100%
src\application.py	OrderService.checkout	6	0	0	100%
src\application.py	*(no function)*	3	0	0	100%
src\domain.py	Order._validate_confirmation	6	0	0	100%
src\domain.py	Order.confirm	4	0	0	100%
src\domain.py	*(no function)*	10	0	0	100%
src\repository.py	SqlOrderRepository.__init__	1	0	0	100%
src\repository.py	SqlOrderRepository.save	1	0	0	100%
src\repository.py	make_engine	3	0	0	100%
src\repository.py	*(no function)*	13	0	0	100%
src\security.py	can_cancel	6	0	0	100%
src\security.py	*(no function)*	2	0	0	100%
Total		**56**	**0**	**0**	**100%**

coverage.py v7.17.0, created at 2026-10-28 19:32 -0600

Coverage report: 100%

Files | Functions | Classes

coverage.py v7.17.0, created at 2026-10-28 19:32 -0600

File ▲	class	statements	missing	excluded	coverage
src__init__.py	*(no class)*	0	0	0	100%
src\application.py	OrderService	7	0	0	100%
src\application.py	*(no class)*	3	0	0	100%
src\domain.py	CustomerHasDebt	0	0	0	100%
src\domain.py	Order	10	0	0	100%
src\domain.py	*(no class)*	10	0	0	100%
src\repository.py	Base	0	0	0	100%
src\repository.py	OrderRow	0	0	0	100%
src\repository.py	SqlOrderRepository	2	0	0	100%
src\repository.py	*(no class)*	16	0	0	100%
src\security.py	*(no class)*	8	0	0	100%
Total		**56**	**0**	**0**	**100%**

coverage.py v7.17.0, created at 2026-10-28 19:32 -0600

La rama aún no probada corresponde al caso en que el parámetro now (En **src/security.py**)no se proporciona. Las pruebas actuales siempre envían una fecha explícita para hacer los resultados deterministas. Por ello, no se ha verificado el camino alternativo en el que **can_cancel()** utiliza la fecha y hora actual por defecto. Esta rama podría probarse con una prueba que omita el argumento now, aunque sería necesario diseñarla cuidadosamente porque dependería del reloj del sistema.

Evidencia 8

Github repo: https://github.com/Lamaguer/NovaRent-U5



El pipeline detectó que una prueba falló porque la aserción esperaba un resultado incorrecto. En **test_stranger_denied**, se cambió temporalmente assert not por assert. La función **can_cancel()** devolvió False, como corresponde a un usuario que no es propietario, pero la prueba esperaba True. Por eso pytest reportó un AssertionError y GitHub Actions marcó el workflow como fallido.

Evidencia 9

CSV en Tabla para mejor visibilidad

ID	Regla	Entrada	Resultado esperado	Tipo	Prueba	Resultado real
CP-01	RN-02 Requisitos de entrega	Licencia, ID y depósito $1000/1000$ OK	Renta pasa a ENTREGADO	Éxito	test_cp01_entrega_con_requisitos_completos	PASSED
CP-02	RN-02 Requisitos de entrega	Sin licencia vigente	ReglaNegocioError RN-02; sigue PENDIENTE	Error	test_cp02_entrega_sin_licencia_rechazada	PASSED
CP-03	RN-02 Requisitos de entrega	Depósito pagado = requerido ($1000.00)	Entrega permitida (>=)	Límite	test_cp03_deposito_exacto_es_valido	PASSED
CP-04	RN-02 Requisitos de entrega	Depósito pagado $999.99 de $1000.00	Rechazo RN-02	Límite/Error	test_cp04_deposito_un_centavo_menos_rechazado	PASSED
CP-05	RN-03 Extensión con capacidad	Capacidad 2, 1 traslape, nueva fin 15-nov	Periodo extendido	Éxito	test_cp05_extension_con_capacidad	PASSED
CP-06	RN-03 Extensión con capacidad	Capacidad 1, 1 traslape	Rechazo RN-03; fecha fin sin cambios	Error	test_cp06_extension_sin_capacidad_rechazada	PASSED
CP-07	RN-03 Extensión con capacidad	Otra renta inicia 15-nov = nueva fin	Sin traslape (intervalo semiabierto); permitido	Límite	test_cp07_otra_renta_inicia_justo_al_nuevo_fin	PASSED

CP-08	RN-03 Extensión con capacidad	Nueva fin = fin actual	Rechazo: debe ampliar	Límite/Error	test_cp08_extension_que_no_amplia_rechazada	PASSED
CP-09	RN-01 Upgrade conserva tarifa	ECONOMICO -> SUV, tarifa $500	Categoría SUV; tarifa $500	Éxito	test_cp09_upgrade_conserva_tarifa	PASSED
CP-10	RN-01 Upgrade conserva tarifa	SUV -> ECONOMICO	Rechazo (no es superior)	Error	test_cp10_upgrade_a_categoria_inferior_rechazado	PASSED
CP-11	RN-01 Upgrade conserva tarifa	ECONOMICO -> COMPACTO (siguiente nivel)	Permitido; tarifa $500	Límite	test_cp11_upgrade_a_la_siguiente_categoria	PASSED
CP-12	RN-01 Upgrade conserva tarifa	SUV -> SUV	Rechazo (misma categoría)	Límite/Error	test_cp12_upgrade_a_misma_categoria_rechazado	PASSED
CP-13	Persistencia (commit)	Guardar renta con upgrade, commit, releer	Se recupera SUV y $500.00 exacto	Integración	test_cp13_persistencia_commit_conserva_tarifa_tras_upgrade	PASSED
CP-14	Persistencia (rollback)	Guardar y rollback	get() devuelve None	Integración	test_cp14_persistencia_rollback_no_guarda	PASSED
CP-15	Mock pasarela de depósito	Pasarela responde True	Entrega; retener depósito llamado 1 vez con $1000	Mock/Éxito	test_cp15_mock_deposito_aprobado	PASSED

CP-16	Mock pasarela de depósito	Pasarela responde False	RuntimeError; renta sigue PENDIENTE	Mock/ Error	test_cp16_mock_deposito_rechazado_no_entrega	PASSED
CP-17	Mock pasarela de depósito	Requisitos incompletos	No se llama a la pasarela	Mock/ Límite	test_cp17_requisitos_incompletos_no_cobran	PASSED

Evidencia 9 - Pruebas ejecutables

```python
# ---------- RN-02 Requisitos de entrega ----------
def test_cp01_entrega_con_requisitos_completos():
    r = renta()
    r.entregar(req())
    assert r.estado == "ENTREGADO"


def test_cp02_entrega_sin_licencia_rechazada():
    r = renta()
    with pytest.raises(ReglaNegocioError, match="RN-02"):
        r.entregar(req(lic=False))
    assert r.estado == "PENDIENTE"


def test_cp03_deposito_exacto_es_valido():
    r = renta()
    r.entregar(req(pagado="1000.00", requerido="1000.00"))
    assert r.estado == "ENTREGADO"


def test_cp04_deposito_un_centavo_menos_rechazado():
    r = renta()
    with pytest.raises(ReglaNegocioError, match="RN-02"):
        r.entregar(req(pagado="999.99"))
    assert r.estado == "PENDIENTE"
```

```python
# ---------- RN-03 Extension requiere disponibilidad ----------
def test_cp05_extension_con_capacidad():
    r = renta()
    otro = PeriodoRenta(date(2026, 11, 5), date(2026, 11, 12))
    r.extender(date(2026, 11, 15), inv(2, otro))
    assert r.periodo.fecha_fin == date(2026, 11, 15)


def test_cp06_extension_sin_capacidad_rechazada():
    r = renta()
    otro = PeriodoRenta(date(2026, 11, 5), date(2026, 11, 20))
    with pytest.raises(ReglaNegocioError, match="RN-03"):
        r.extender(date(2026, 11, 15), inv(1, otro))
    assert r.periodo.fecha_fin == date(2026, 11, 10)


def test_cp07_otra_renta_inicia_justo_al_nuevo_fin():
    r = renta()
    otro = PeriodoRenta(date(2026, 11, 15), date(2026, 11, 20))
    r.extender(date(2026, 11, 15), inv(1, otro))
    assert r.periodo.fecha_fin == date(2026, 11, 15)


def test_cp08_extension_que_no_amplia_rechazada():
    r = renta()
    with pytest.raises(ReglaNegocioError):
        r.extender(date(2026, 11, 10), inv(5))


# ---------- RN-01 Upgrade conserva tarifa original ----------
def test_cp09_upgrade_conserva_tarifa():
    r = renta("ECONOMICO")
    r.aplicar_upgrade("SUV")
    assert r.categoria_asignada == "SUV"
    assert r.tarifa_original == D("500.00")
```

```python
def test_cp10_upgrade_a_categoria_inferior_rechazado():
    r = renta("SUV")
    with pytest.raises(ReglaNegocioError):
        r.aplicar_upgrade("ECONOMICO")
    assert r.categoria_asignada == "SUV"


def test_cp11_upgrade_a_la_siguiente_categoria():
    r = renta("ECONOMICO")
    r.aplicar_upgrade("COMPACTO")
    assert r.tarifa_original == D("500.00")


def test_cp12_upgrade_a_misma_categoria_rechazado():
    with pytest.raises(ReglaNegocioError):
        renta("SUV").aplicar_upgrade("SUV")


def test_cp12b_categoria_desconocida_rechazada():
    with pytest.raises(ReglaNegocioError, match="desconocida"):
        renta().aplicar_upgrade("LIMUSINA")


def test_periodo_invalido_rechazado():
    with pytest.raises(ReglaNegocioError):
        PeriodoRenta(date(2026, 11, 5), date(2026, 11, 5))


def test_entregar_dos_veces_rechazado():
    r = renta()
    r.entregar(req())
    with pytest.raises(ReglaNegocioError, match="PENDIENTE"):
        r.entregar(req())
```

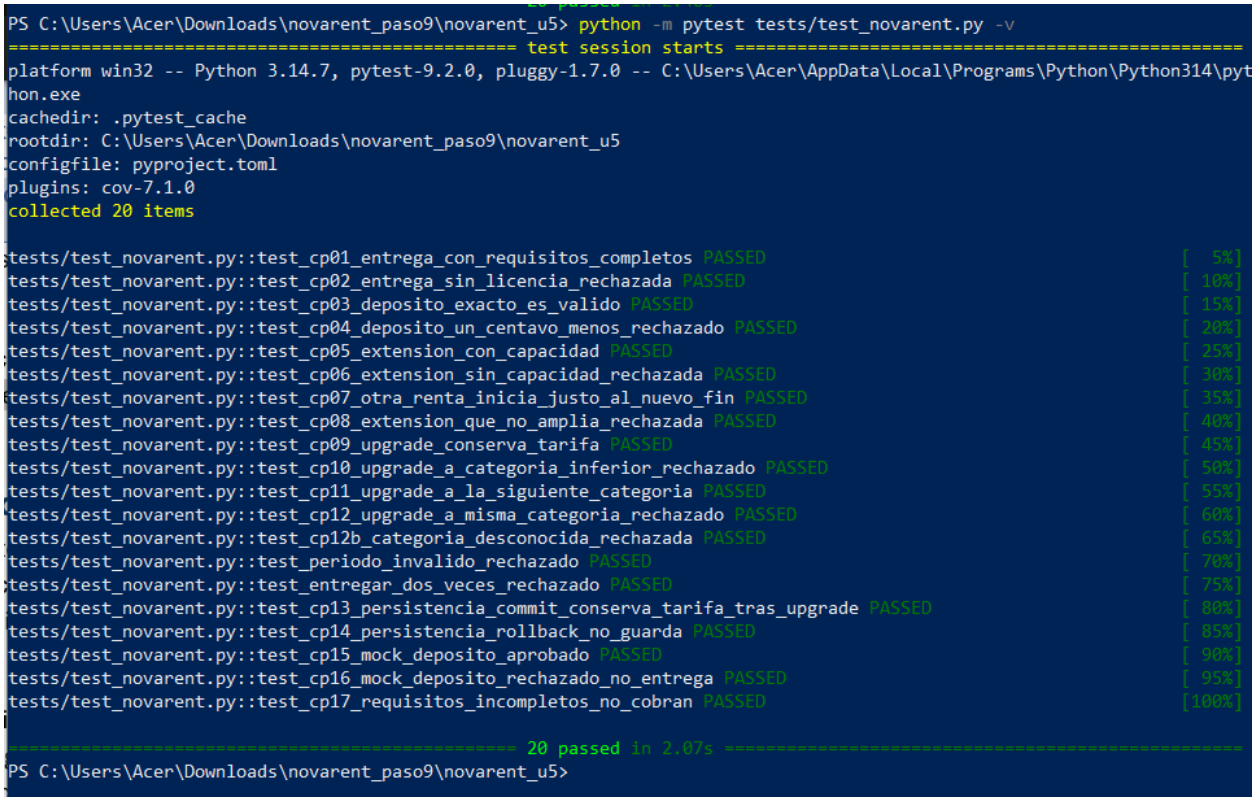
```python
# ---------- Persistencia (SQLite en memoria) ----------
def test_cp13_persistencia_commit_conserva_tarifa_tras_upgrade():
    engine = make_engine()
    r = renta("ECONOMICO")
    r.aplicar_upgrade("SUV")
    with Session(engine) as s:
        SqlRentaRepository(s).save(1, r)
        s.commit()
    with Session(engine) as s:
        g = SqlRentaRepository(s).get(1)
        assert (g.categoria_asignada, g.tarifa_original) == ("SUV", D("500.00"))


def test_cp14_persistencia_rollback_no_guarda():
    engine = make_engine()
    with Session(engine) as s:
        SqlRentaRepository(s).save(1, renta())
        s.rollback()
        assert SqlRentaRepository(s).get(1) is None


# ---------- Mock: pasarela de deposito (adaptador simulado) ----------
def test_cp15_mock_deposito_aprobado():
    pasarela = Mock()
    pasarela.retener_deposito.return_value = True
    r = renta()
    assert ServicioEntrega(pasarela).entregar(r, req()) == D("1000.00")
    pasarela.retener_deposito.assert_called_once_with(D("1000.00"))
    assert r.estado == "ENTREGADO"

def test_cp16_mock_deposito_rechazado_no_entrega():
    pasarela = Mock()
    pasarela.retener_deposito.return_value = False
    r = renta()
    with pytest.raises(RuntimeError, match="deposito rechazado"):
        ServicioEntrega(pasarela).entregar(r, req())
    assert r.estado == "PENDIENTE"


def test_cp17_requisitos_incompletos_no_cobran():
    pasarela = Mock()
    with pytest.raises(ReglaNegocioError):
        ServicioEntrega(pasarela).entregar(renta(), req(ident=False))
    pasarela.retener_deposito.assert_not_called()
```



Decisiones de diseño

Se seleccionaron las Reglas RN01-RN03, al ser estas reglas sumamente importantes para el modelo del negocio (Esto no significa que las demas no importan claro), cubren riesgos distintos, como cobros incorrectos (RN-1), entregas indebidas (RN-02) y sobreventa de inventario (RN-03), este tipo de tests se sintieron más fácil de implementar por su similitud a lo visto dentro del laboratorio.

Para cada regla se definió un caso de éxito, uno de error y al menos uno de límite, esto se hizo para poder tener tests de situaciones variadas ya que es más adecuado tener una gran variedad en estos tests para poder prevenir cualquier tipo de problema dentro del modelo de la empresa, si solo se cubrieran tests de éxito no se tendría tanta certeza que los casos de error sucedan correctamente.

Se usan datos Decimales para evitar errores de redondeo de dinero con float.

Los periodos fueron definidos como (Inicio, fin), de esta forma una renta que termina en el día 15 no choca con una que empieza en el día 15.

Se usa SQLite en memoria con SQLAlchemy para que cada prueba cree su propia base y no toque datos reales del sistema, cada tarifa se guarda con Decimal.
Con estas decisiones de diseño se alcanzó una cobertura del 100% en las líneas sobre src/. Esto no significa que sea libre de errores.
Vibe Coding Corrección - La IA es una herramienta muy inteligente por lo que al analizar sus resultados y casos dados resultó muy difícil poder argumentar el contrario de sus decisiones, aunque esta no está libre de errores, al querer crear ciertos tests en .py files separadas cuando estos tests podrían simplemente ir en el mismo que testea Nova Rent en sí, esto se corrigió para tener una mejor organización por el momento.
La IA aconsejó programar en JAVA por su POO pero decidimos hacerlo con Python gracias a que el laboratorio es en Python por lo que sentimos que seguir el camino del laboratorio era la mejor opción disponible.

Cierre

Muchas de las pruebas de los pedidos serian terribles si no funcionaran, unas más que otras claro, no pasaría un daño extremadamente grave si el administrador podrá cancelar órdenes en estado SHIPPED, sentimos que la peor prueba que podría fallar sería la del VIP ya que si a las personas con VIP no se les aplicará el descuento el punto entero del VIP no tendría sentido y sería faltarle el respeto y perder la confianza de estos usuarios que probablemente no vuelvan a ser usuarios VIP, y hasta podría llegar a un punto de demanda por cuanto dinero se estaría “robando” la empresa por no aplicar esos descuentos del 15%, pero gracias a pruebas como **test_vip_discount()** nos podemos asegurar que estos descuentos si lleguen a pasar sin problema alguno.
Otro defecto catastrófico sería si cualquier usuario fuera capaz de eliminar o cancelar la orden ajena, esto sería un gran problema ya que claramente ese no se supone que pase, pero gracias a la prueba **test_stranger_denied()** no nos tenemos que preocupar de que eso pueda llegar a pasar, por el momento.